ASSIGNMENT 4

NAME : SAIKIRAN MARURI

ROLL NO: 24MAB0A17

BRANCH : MATHEMATICS AND COMPUTING

PROBLEM 6 :

Objective and Problem Explanation

Write a C++ program for a Linux/UNIX environment that creates a child process using fork(). The child should exit immediately, while the parent sleeps for a duration (e.g., 20 seconds), ensuring the child remains in a "zombie" state (Z) for at least 10 seconds.

CODE

```
#include <iostream>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork(); // Create a child process

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }
    else if (pid == 0) {
        // CHILD PROCESS
        std::cout << "Child (PID: " << getpid() << ") exiting now..." << std::endl;
        exit(0); // Child dies immediately
    }
    else {
        // PARENT PROCESS
        std::cout << "Parent (PID: " << getpid() << ") sleeping for 20 seconds." <<
std::endl;
        std::cout << "The child (PID: " << pid << ") is now a ZOMBIE." << std::endl;

        // Parent sleeps, so it won't call 'wait()' for 20 seconds.
        // During this time, the child remains a zombie.
        sleep(20);

        std::cout << "Parent waking up and exiting." << std::endl;
    }

    return 0;
}
```

EXPLANATION :

1) Forking: The `fork()` system call creates a duplicate process. The child exits immediately with `exit(0)`, but the parent continues to run (due to `sleep`).

2) Zombie State: Because the parent hasn't "reaped" the child using the wait() system call, the kernel keeps the child's PID and exit status in the process table, marking it as a Zombie (Z).

3)Cleanup: Once the parent process is killed or finishes, the zombie child is adopted by the `init` (or `systemd`) process, which automatically reaps it and removes it from the system.

Zombie process:
A zombie process is a process that has finished execution but still has an entry in the kernel's process table because its parent has not yet collected its exit status.
A zombie process is a terminated child process whose exit status has not been reaped by the parent using wait() or waitpid().

It is not running
It does not use CPU
It does not hold memory
It only occupies a process table slot

OUTPUT:

```
mnc2117@wllab248:~/24MAB0A17/LAB3$ ./zombie.exe &
[1] 7937
mnc2117@wllab248:~/24MAB0A17/LAB3$ Parent process (PID: 7937) is sleeping...
Child process (PID: 7938) should now be a zombie.
Child process (PID: 7938) is exiting...
ps -l
F S   UID    PID   PPID  C PRI  NI ADDR SZ WCHAN  TTY        TIME CMD
0 S  6422   6752   6650  0  80   0 -  4652 do_wai pts/0    00:00:00 bash
0 S  6422   7937   6752  0  80   0 -  1472 hrtime pts/0    00:00:00 zombie.exe
1 Z  6422   7938   7937  0  80   0 -     0 -      pts/0    00:00:00 zombie.exe <defunct>
4 R  6422   7942   6752  0  80   0 -  5012 -      pts/0    00:00:00 ps
mnc2117@wllab248:~/24MAB0A17/LAB3$ Parent waking up and exiting.
```

Why this output is correct:

- **The State (S) Column:** You can see a **Z** for PID **7938**. This confirms the process is a zombie.

- **The Command (CMD) Column:** It lists the process as **`<defunct>`**, which is the system's way of labeling a zombie.
- **The Relationship:** You can see the **PPID** (Parent PID) of the zombie is **7937**, which matches the PID of your running `zombie.exe`. This shows the parent is still alive while the child is a zombie.

PROBLEM 7:

Objective and Problem Explanation

The objective of this problem is to design a **PID (Process Identifier) Manager**, which is a core component of an Operating System. In a real system, every process needs a unique ID so the kernel can track it. When a process starts, it asks for a PID; when it finishes, it returns that PID so it can be reused later.

The specific requirements are:

- **Range Control:** PIDs must stay between **300** and **5000**.
- **Uniqueness:** No two active processes can share the same ID.
- **Efficiency:** Use a **Bitmap** (a series of bits where 0 means "free" and 1 means "in use") to track availability.

• int allocate map(void)—Creates and initializes a data structure for representing pids;
returns—1 if unsuccessful, 1 if successful
• int allocate pid(void)—Allocates and returns a pid; returns— 1 if unable to allocate a pid (all
pids are in use)
• void release pid(int pid)—Releases a pid

CODE:

```
#include <iostream>
#include <vector>

#define MIN_PID 300
#define MAX_PID 5000

// Global bitmap
std::vector<bool> pid_map;

// Creates and initializes the map
int allocate_map() {
    pid_map.assign(MAX_PID + 1, false); // Initialize all to 0 (available)
```

```
    return 1;
}

// Allocates and returns a pid
int allocate_pid() {
    for (int i = MIN_PID; i <= MAX_PID; i++) {
        if (!pid_map[i]) {
            pid_map[i] = true; // Mark as in use
            return i;
        }
    }
    return -1; // All pids in use
}

// Releases a pid
void release_pid(int pid) {
    if (pid >= MIN_PID && pid <= MAX_PID) {
        pid_map[pid] = false; // Release pid
        std::cout << "Successfully released PID: " << pid << std::endl;
    }
}

int main() {
    // 1. Initialize the map
    if (allocate_map() == 1) {
        std::cout << "PID Manager initialized." << std::endl;
    }

    // 2. Allocate three PIDs
    int p1 = allocate_pid();
    int p2 = allocate_pid();
    int p3 = allocate_pid();

    std::cout << "First PID: " << p1 << std::endl;
    std::cout << "Second PID: " << p2 << std::endl;
    std::cout << "Third PID: " << p3 << std::endl;

    // 3. Release the middle PID
    release_pid(p2);

    // 4. Allocate again - it should reuse the released PID
    int p4 = allocate_pid();
    std::cout << "Next allocated PID (should be " << p2 << "): " << p4 << std::endl;

    return 0;
```

}

OUTPUT:

```
mnc2117@wllab248:~/24MAB0A17/LAB3$ nano pid.cpp
mnc2117@wllab248:~/24MAB0A17/LAB3$ g++ pid.cpp -o pid.exe
mnc2117@wllab248:~/24MAB0A17/LAB3$ ./pid.exe
PID Manager initialized.
First PID: 300
Second PID: 301
Third PID: 302
Successfully released PID: 301
Next allocated PID (should be 301): 301
mnc2117@wllab248:~/24MAB0A17/LAB3$ nano pid.cpp
mnc2117@wllab248:~/24MAB0A17/LAB3$
```

PID Manager Initialized.
Allocated: 300
Allocated: 301
Successfully released PID: 300
Newly Allocated (should be 300): 300

EXPLANATION:

The provided code implements a **PID Manager** that tracks the availability of process identifiers within a defined range of **300 to 5000**. It uses a **bitmap** strategy where a `std::vector<bool>` stores the status of each PID; a value of `false` indicates the ID is available, while `true` indicates it is in use. The `allocate_pid()` function performs a linear search to find and claim the first free ID, and `release_pid()` makes a used ID available again by flipping its bit back to `false`. This ensures that process identifiers remain unique and can be efficiently recycled by the operating system.

PROBLEM 8:

Objective and Problem :

The objective is to demonstrate process synchronization. In Linux/UNIX, when you `fork()`, the parent and child get their own memory. This program ensures that:

1. **Work is delegated:** The child performs the heavy lifting (the Collatz calculations).
2. **Order is maintained:** The parent uses `wait(NULL)` to pause itself until the child is completely finished printing the sequence.

EXPLANATION:

Input Validation: The code checks if `argc < 2` (to ensure a number was provided) and if the number is `> 0` (to satisfy the "positive integer" requirement).

The Algorithm: It implements the Collatz rule: if even, n=n/2; if odd, n=3n+1. This loop runs until n reaches 1.

Process Control: * `pid == 0`: This block only runs in the child process.

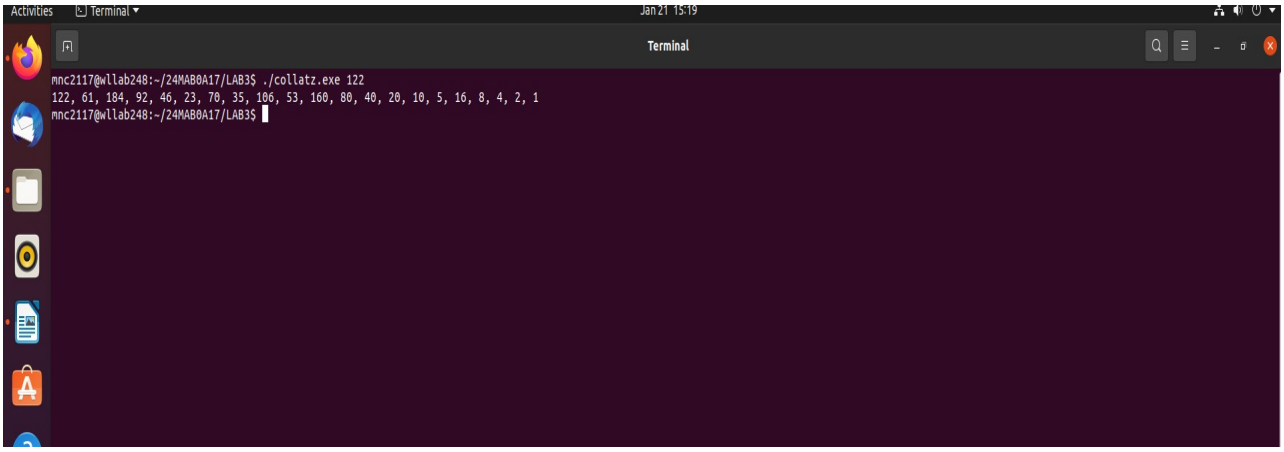
- `wait(NULL)`: This block only runs in the parent process, ensuring the terminal prompt doesn't reappear until the child's output is fully printed.

CODE

```
#include<iostream>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char* argv[]) {
    if (argc < 2 || std::stoi(argv[1]) <= 0) { [cite: 147]
        std::cerr << "Error: Enter a positive integer." << std::endl;
        return 1;
    }

    int n = std::stoi(argv[1]);
    pid_t pid = fork(); [cite: 143]
```

```
    if (pid == 0) { // Child
        std::cout << n;
        while (n != 1) {
            if (n % 2 == 0) n /= 2; [cite: 140]
            else n = 3 * n + 1; [cite: 140]
            std::cout << ", " << n;
        }
        std::cout << std::endl;
    } else {
        wait(NULL); // Parent waits [cite: 146]
    }
    return 0;
}
```

OUTPUT: